9
Nearest Neighbor and Normalization

The two topics of this chapter are distinct. Data normalization is important not just for nearest neighbor, and nearest neighbor is not the only method than can benefit from normalization. However, the effects and significance of normalization are most clearly illustrated in the context of the nearest neighbor classifier, so they are presented together.

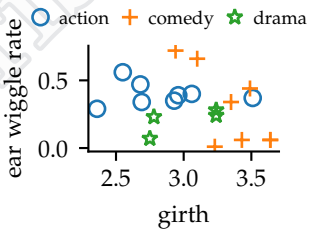

Figure 9.1: Data for 20 hippopotamuses with two features: x_1 is the hippopotamus's girth and x_2 is the hippopotamus's ear wiggle rate. The associated label (y) is the hippopotamus's favorite genre of movie (shown by color and shape).

Nearest Neighbor

If presented with a new (testing) example for classification, perhaps the most obvious way of using the training data for prediction would be to find the training example that is most similar to this new example and predict that this new example's class is the same as "its similar friend" from the training set. If we define similarity as inversely related to distance, this is the essential idea behind nearest neighbor classification.

Euclidean distance is the most straight-forward way to measure the (dis-)similarity between two examples' feature vectors. If $\vec{x}$ and $\vec{x}'$ are two feature vectors (either could be a training or testing point), the Euclidean distance between them is

$$\begin{aligned} \|\vec{x} - \vec{x}'\| &= \sqrt{\sum_{j=1}^{n} \left(x_j - x'_j\right)^2} \\ &= \sqrt{(\vec{x} - \vec{x}')^\intercal (\vec{x} - \vec{x}')} \,. \end{aligned} \tag{9.1}$$

Consider the dataset shown graphically in Figure 9.1. This is a classification problem, so we are trying to predict y, the hippopotamus's favorite movie genre which can be either "action," "comedy," or "drama." Figure 9.2 shows an example prediction using the nearest neighbor classifier. Our new hippopotamus for whom we would like to predict her favorite movie type has girth of 3.3 and an ear wiggle rate of 0.5. This point is shown as a black "x." The nearest

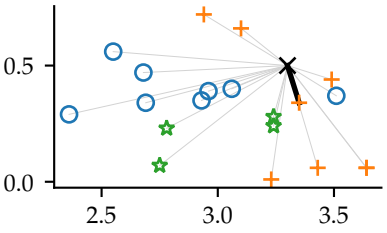

Figure 9.2: Nearest neighbor classification of black "x" testing point. The point is compared to each member in the training set. The closest training point is comedy (orange +) and therefore classifier returns "comedy."

neighbor classifier predicts her favorite movie genre is "comedy" because the label of the closest point in the training set is "comedy."

Lazy Evaluation

Nearest neighbor (along with its variants discussed below) is a **lazy method**. Nothing (or very little) is done at training time. The dataset is stored, but no weights, parameters, or other values are calculated. Instead, the work occurs during testing time. The testing point is compared to each point in the training set to find the closest.

By contrast, an **eager method** like logistic regression calculates weights at training time, iterating over each element in the training set, possibly multiple times. After training, the training set can be forgotten. Testing just involves the weights and the testing example.

Each has advantages and drawbacks, depending on their application. If only a few points will be queried and the time for their queries is not as critical, a lazy method might be preferred. If many points will be tested or query time is more critical (and more computation can be afforded during training), a eager method might be preferred. We return to how to speed up nearest neighbor queries (at the expense of training time) later.

For now, just note that nearest neighbor operates differently than the other methods we discuss: There is no "training" or "learning" algorithm. Rather, all of the work is done at testing time.

Decision Boundary

If we were to query every possible testing point and color the location with the label nearest neighbor would return, we would get Figure 9.3. While not practical in general, for a small toy example it provides a helpful visualization. The black lines show the decision boundary between classes. They are exactly equidistant from two training points. Figure 9.4 shows an example with three features. In all cases, the boundaries will be piece-wise linear, composed of flat sections (lines segments or polygons in these figures).

In contrast to methods in previous chapters, the decision boundaries of nearest neighbor are jagged and complicated. Logistic regression states that $\tilde{f}(\vec{x})$ is a linear function of $\vec{x}$ and therefore the decision boundary is linear. Although neural networks produce complex decision boundaries, before learning starts the parametric form of that decision boundary is fixed. These methods are all **parametric**. By contrast, nearest neighbor is **non-parametric**. The complexity of the resulting rule (or decision boundary) can grow arbitrarily large as the number of data points in the training set grows.

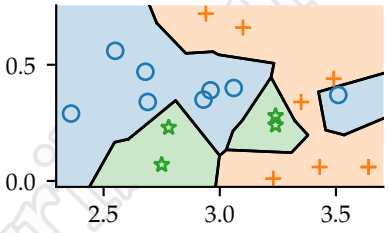

Figure 9.3: Decision boundary for nearest neighbor shown in black.

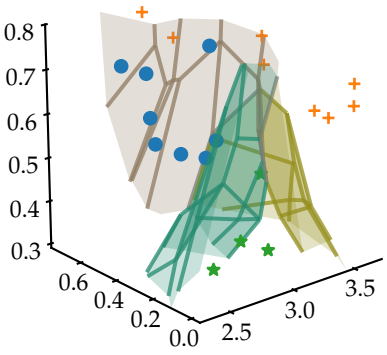

Figure 9.4: Decision boundary for nearest neighbor for a slightly modified version of the same dataset, where an additional feature x_3 (fraction of the day spent sleeping) has been added. Boundaries between classes are piecewise linear and shown.

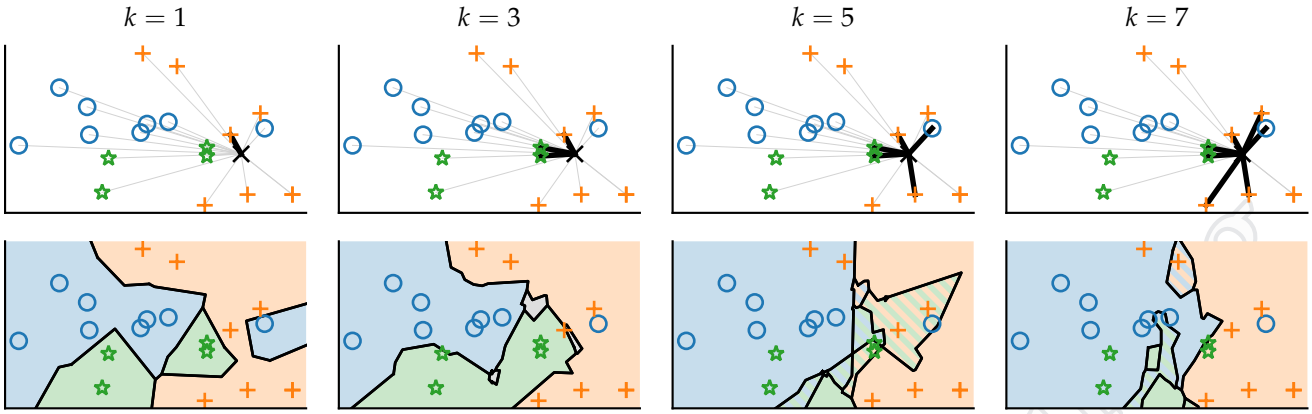

Figure 9.5: Comparison of $k \in \{1,3,5,7\}$. (Top) calculation for a specific point. For k=1 and k=7, comedy (orange +) is predicted for this point. For k=3, drama (green star) is predicted. There is a tie for k=5 (between comedy and drama); either of the tied labels could be predicted. (Bottom) the resulting classification for every point in the space. Gray areas are where all three classes are tied. Striped areas are where two classes are tied.

k-Nearest Neighbor

The non-parametric complexity of the nearest neighbor classifier means that it frequently overfits. Consider the right side of Figure 9.3. A single blue (y = action) training point has caused a large region of possible testing points to be classified as action, despite that most of the other points in the area are comedy (orange +). To combat this, we might consider looking beyond the nearest neighbor to consider other points in the area. Setting a fixed radius within which to consider training points is problematic, because we have to know how large of a radius to set to insure at least one training point falls inside the circle or sphere. Instead, we fix the number of nearest points to consider.

The k-nearest neighbor classifier has the hyper-parameter k which sets the number of points to consider. If $k = 1$, k-nearest neighbor is the same as nearest neighbor. When k is larger than 1, k-nearest neighbor finds the k closest training points to the testing point. It reports the plurality label among the k training points.

The algorithm is shown in Algorithm 9.6. Note that we calculate the square of the distances and skip the square-root in Equation 9.1 because it doesn't affect the ordering of the closest points. Once the distances from the testing point ($\vec{x}$) to each of the training points ($\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_m$) are calculated, there are many ways to find the most frequent label among the points corresponding to the k closest distances. A simple way is to sort the distances, keeping track of the

KNNPREDICT
In: $\boldsymbol{X}, \boldsymbol{Y}, k, \vec{x}$
Out: $f(\vec{x})$

for all $i \in \{1, \ldots, m\}$ **do**
  $d_i \leftarrow (\vec{x}_i - \vec{x})^\intercal (\vec{x}_i - \vec{x})$
Let i_j be the index of the jth smallest value of $d_1, \ldots, d_m$
Let f be the most frequent value in $y_{i_1}, y_{i_2}, \ldots, y_{i_k}$
return f

Algorithm 9.6: k-nearest neighbor prediction algorithm

original indices. But, it is faster to do a linear scan of distances, keeping track just of the k smallest ones found so far (and their indices). If there is not a unique most-frequent class, any of the most-frequent classes may be returned using an arbitrary tie-breaking method. This can happen if k is even or if there are more than 2 classes.

Figure 9.5 shows the effect on this small dataset of changing k. Although not entirely clear from these plots, as k increases, the classifier becomes "simpler" in that it is not as dependent on one (or a few) of the training data points. For example, when k=13 for this small dataset, the decision surface is close to linear, as shown in Figure 9.7 In the extreme case when k=m, the k-nearest neighbor method predicts the most common class in the training set, regardless of the query point $\vec{x}$.

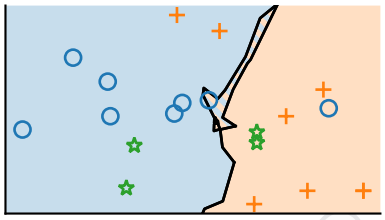

Figure 9.7: k-nearest neighbor for k=13

Consistency of Nearest Neighbor

Consider when k=1 (the "plain" nearest neighbor algorithm). Because of the flexibility of the resulting classifier, it is tempting to think that in the limit of infinite data ($m \rightarrow \infty$) the classifier will converge to the **Bayes-optimal classifier**.

Key point
Nearest neighbor is not consistent.

A method that converges to the Bayes-optimal classifier in the limit of infinite data is called **consistent**. Unfortunately, *nearest neighbor is not consistent*. For nearest neighbor to be the same as the **Bayes-optimal classifier**, it must always report the most frequent (or likely) class associated with any point $\vec{x}$. If m=∞, then there are an infinite number of training points essentially right next to $\vec{x}$. Because they are arbitrarily close, their labels are all drawn from $\Pr(y \mid \vec{x})$. Thus, while the label of the closest training point is most probably drawn from the probable class, it is not guaranteed to be.

An example with two classes (C=2), class A and class B: Consider a query hippopotamus with girth 3.2 and ear wiggle rate of 0.5. Thus, the testing point is $\vec{x} = \begin{bmatrix} 3.2 \\ 0.5 \end{bmatrix}$. Assume that for the (true) distribution of classes for hippopotamuses with these measurements, 75% of them are of class A and 25% of them are of class B. Then the Bayes-optimal classifier would report "class A" because it is more likely and would have an error rate of 25% $=$ 0.25 because one-quarter of the time this would be wrong.

Now consider a (k=1) nearest neighbor classifier with an infinite amount of training data. Provided the same testing point, it would look for the closest example in the training data. Because there are an infinite number of hippopotamuses in the training data, this nearest hippopotamus would have essentially the same measurements as the testing hippopotamus. Thus, its label (or class) would be drawn from the same distribution. 75% of the time, it would be of class

A, and therefore nearest neighbor would report "class A" and it would be wrong 25% of the time (because this particular testing hippopotamus is class B 25% of the time). Similarly, the other 25% of the time, the nearest neighbor would be of class B, and therefore nearest neighbor would report "class B" and it would be wrong 75% of the time. Taken together, the total expected error rate would be $0.75 \times 0.25 + 0.25 \times 0.75 = 0.375 = 37.5\%$, which is higher than the **Bayes-optimal error rate** of 25%.

More generally, the error of nearest neighbor at point $\vec{x}$ *when* $m{=}\infty$ (the asymptotic error rate) is

$$\text{err}_{\text{NN}}(\vec{x}) = \sum_{c=1}^{C} \Pr(y = c \mid \vec{x})\,(1 - \Pr(y = c \mid \vec{x})) \qquad (9.2)$$

There are C possible classes.

which is the sum over each of the classes, c, of the chance the nearest neighbor is of this class, $\Pr(y = c \mid \vec{x})$, times the probability the testing point is not of this class, $1 - \Pr(y = c \mid \vec{x})$. The error rate of the entire classifier is the mean of this rate over $\vec{x}$, where the mean is take with respect to the true distribution $\Pr(\vec{x})$.

In general, how err_{NN} compares with $\text{err}_{\text{Bayes}}$, the Bayes-optimal error rate, depends on the true distribution, which is not generally knowable. Further, err_{NN} (at least as discussed here) is for an infinite training set.

Given these caveats, we can say a few things. If the Bayes-optimal error rate is 0, then nearest neighbor converges (as $m \to \infty$) to the Bayes-optimal classifier. This is true when, at every point $\vec{x}$, the class y is a deterministic function of $\vec{x}$. So, if a hippopotamus's girth and ear wiggle rate are entirely predictive of her favorite genre of movie, then nearest neighbor will converge to the optimal classifier (one with no error) in the limit of infinite training data. On the other side, if there is *no* relationship between $\vec{x}$ and y (knowing $\vec{x}$ does not help at all in determining y) and all classes are equally likely, then any guess is as good as any other and the Bayes-optimal classifier and nearest neighbor (along with any other method) will produce the same error rate, $\frac{C-1}{C}$.

More generally, Figure 9.8 shows the asymptotic ($m{=}\infty$) worst-case nearest neighbor error rate as a function of the Bayes-optimal error rate. The asymptotic nearest neighbor error rate is never greater than twice the Bayes-optimal rate: $\frac{\text{err}_{\text{NN}}}{\text{err}_{\text{Bayes}}} \leq 2$. Further, it is never greater than 0.25 more than the Bayes-optimal rate: $\text{err}_{\text{NN}} - \text{err}_{\text{Bayes}} \leq 0.25$.

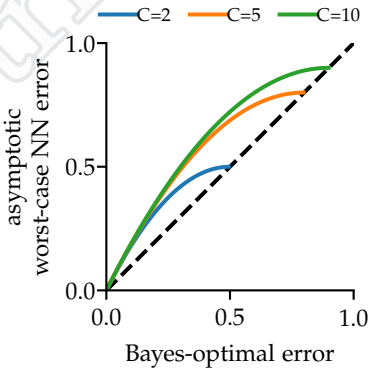

Figure 9.8: Worst-case asymptotic ($m{=}\infty$) error rate for nearest neighbor, as a function of the Bayes-optimal error rate for the same problem. The dashed line is where the NN error rate equals the Bayes-optimal error rate. Lines only extend to an error rate of $\frac{C-1}{C}$ because that is the maximal Bayes-optimal error rate.

Consistency of k-Nearest Neighbor

Although nearest neighbor is not consistent, k-nearest neighbor can be. The important factor is how to pick k. The problem with nearest

neighbor was that the closest point might not be from the most common class for the testing point $\vec{x}$. So, we need to increase k to query more points to find the true most common class. However, if k is too large, then we are using training points from too far away, where $\Pr(y \mid \vec{x})$ is different.

Consistency is about when $m \to \infty$. Therefore, we need to establish a rule for how k will change with m, so that we can reason about the limit as $m \to \infty$. We need k to grow, so that we consider more and more points in the area to ensure we have the true most common class. We also need to consider a smaller and smaller *fraction* of all of the points, so that the region we are considering shrinks around $\vec{x}$ so that all of the training points considered come from the same distribution over the class, y.

Key point
k-nearest neighbor is consistent if k is chosen correctly as a function of m.

Taken together, this means we have two conditions for k as a function of m:

$$\begin{aligned} \lim_{m\to\infty} k(m) &= \infty \\ \lim_{m\to\infty} \frac{k(m)}{m} &= 0 \ . \end{aligned} \tag{9.3}$$

Many rules for picking k satisfy these conditions. For instance, we could pick $k = \sqrt{m}$ or $k = \log m$. Any such rule will assure that k-nearest neighbor is a consistent classifier.

How to Pick k

While it is nice to have a consistent classifier, consistency is a property about when $m \to \infty$. Given that your training set does not have an infinite number of data points, how should be select k? [1]

[1] Alas, there are not an infinite number of hippopotamuses (even for small values of ∞), yet alone the time or space to collect all of their data.

Theory for k-nearest neighbors is not very helpful in this case ($m < \infty$). Therefore, we use hold-out validation or cross-validation. This amounts to extracting a subset of the training set and querying their nearest neighbors in the remaining training set to see how often the most common class of k nearest neighbors in the remaining training set agree with the class of the validation points. Note that the k of k-nearest neighbor is completely different than the k of k-fold cross-validation. It is an unfortunate terminology collision where each technique is well-known for using k to denote an important hyper-parameter.

Leave-one-out (LOO) cross-validation is particularly popular for k-nearest neighbor because of its simplicity. For each point in the training set we find its nearest neighbors in the training set *excluding itself* and check to see if its label agrees with the most common label among those nearest neighbors. This is an example where LOO cross-validation does not take much longer than hold-out validation or general cross-validation.

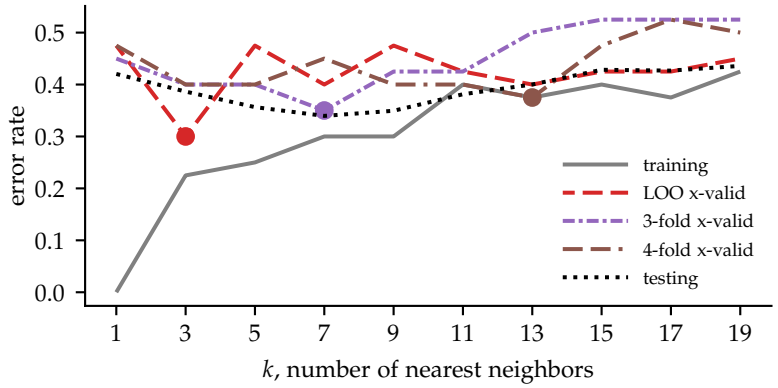

Figure 9.9: Comparison of different validation methods for selecting k for k-nearest neighbor, using the dataset of Figure 9.1, but with twice as many data points. The dotted line is the true error rate (possible to evaluate because in this toy problem, we know the true underlying distribution). Three different cross-validation methods are shown, with the smallest error (and therefore the value of k that would be chosen) shown with a dot.

Figure 9.9 shows an example of using cross-validation to select the number of nearest neighbors, k. Using the training set itself would not work. The lowest error is when k=1 because each point is tested against the training set, which it is already part of, and therefore it is correctly predicted. However, this is not indicative of the error that would be experienced if the classifier were used on testing data. The dotted line is the testing error. For a general problem, it is not knowable. However, because this problem is synthetic and the true distribution $p(\vec{x}, y)$ is known, we can calculate and plot the testing error.

Ideally, our cross-validation method would select k=7, the value that minimizes the testing error. In this case, 3-fold cross-validation does select k=7. There is no way of knowing how many folds of cross-validation will work well. (Without knowing the testing curve, you cannot tell which number of folds is doing better at picking k.) Generally, users of k-nearest neighbor pick a number of folds that is convenient computationally.

Distance Metric

The Euclidean distance metric, Equation 9.1, computes the dissimilarity between two points as the sum of the squares of the differences in each feature. For this reason, it is sometimes referred to as the **ℓ_2-norm**, because of the second power. The resulting nearest

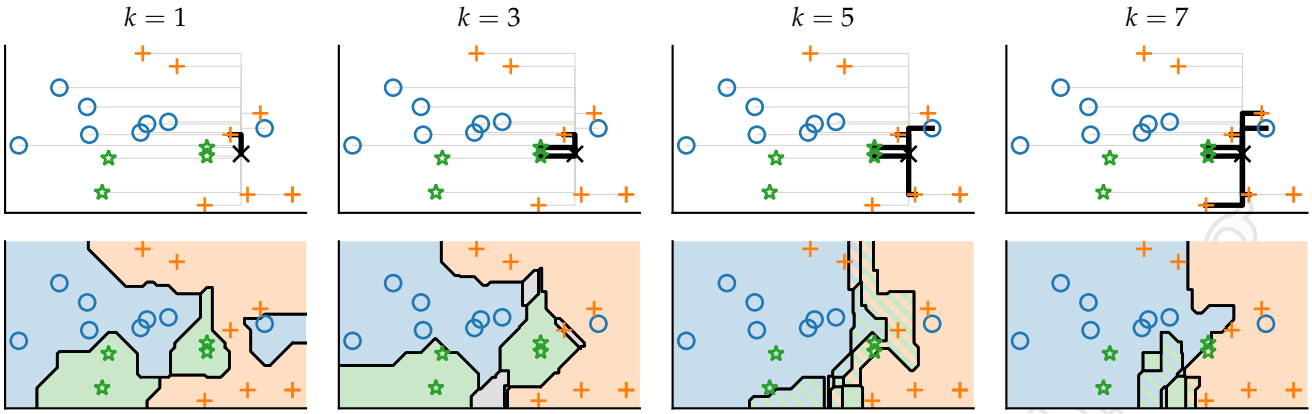

Figure 9.10: The same as Figure 9.5, but for the Manhattan distance, or ℓ_1 norm.

neighbor algorithm is rotationally invariant which means that if all of the data were rotated, the resulting classifier would be rotated by the same amount. This might seems like an advantage, but for many datasets rotating the features is unnatural, and there is no reason to be attached to this property.

Further information
Linear regression and logistic regression are also rotationally invariant.

Further information
Note that if the features are the pixels of an image, rotating the feature vector is *not* the same as rotating the image.

Manhattan Distance

The ℓ_2-norm is not the only choice for nearest neighbor algorithms. We just need *some* method to calculate the dis-similarity between two points. Another common distance metric is the **Manhattan distance**, also known as the **ℓ_1-norm** because it involves the difference (to the "first power") of the feature values:

$$\|\vec{x} - \vec{x}'\|_1 = \sum_{j=1}^{n} \left| x_j - x'_j \right| \ . \tag{9.4}$$

The subscript 1 in $\|\cdot\|_1$ denotes the ℓ_1-norm. A subscript of 2 can be used to denote the ℓ_2-norm, but if no subscript is given, the ℓ_2-norm is assumed.

It is called the Manhattan distance because it would be the distance traveled if one were forced to travel along a grid aligned with the axes, like the streets and avenues of the borough of Manhattan[2]. Figure 9.10 plots k-nearest neighbor for the Manhattan distance metric; compare with Figure 9.5.

[2] Well, if there were no Broadway or other streets that cut diagonally.

Other Distances

While other distance functions are possible to compare two vectors, one advantage of nearest neighbor is that it does not require the data points to be converted into vectors before applying the method. That is, if some method to measure the distance between two x values is provided, nearest neighbor can be applied.

For example, if we wished to classify sentences based on their topic, while we could convert the sentences into a common vector representation, we could also just construct a distance function that worked directly on sentences. One possibility is to use the **edit distance** which is the total number of edits (additions, deletions, or changes) needed to change one sequence of tokens (in this case a sequence of words) into another.

Further information
An edit distance can also be used when the data are DNA sequences, for example.

For other data modalities, other distance measures may be more appropriate. There are measures of the distances between graphs, images, written language, and sounds, just to name a few. To be a valid **distance metric**, a dis-similarity measure between two objects, x and x', must obey four properties:

$$\begin{aligned} d(x,x) &= 0 && \text{distance to self must be 0} \\ d(x,x') &> 0 \text{ if } x \neq x' && \text{distances to other points are positive} \\ d(x,x') &= d(x',x) && \text{symmetry} \\ d(x,x') &\leq d(x,x'') + d(x'',x') && \text{triangle inequality} \end{aligned} \tag{9.5}$$

The **triangle inequality** states that it must be no longer to go directly from x to x' than to go from x to x' through any other point, x''.

Picking a Distance Metric

The choice of which distance metric to use is another hyper-parameter of the k-nearest neighbor method. Knowledge of the domain of the data can help greatly in selecting a distance metric. Certainly if there are similarity or dis-similarity measures that have already been developed for the type of data, these should be considered. Using the "default" Euclidean distance metric is a choice, even if not explicitly made.

However, if application-specific knowledge does not narrow down the possible distance metrics sufficiently, hold-out validation or cross-validation can be used to select the distance metric. In this case, the hyper-parameter is not ordered (like k where the possible values can be sorted). Rather, we just have an unordered collection of choices. Validation checks each possible distance metric to see which performs best on the validation data. For each distance metric, k also needs to be chosen by validation.

For example, if we were considering five choices for k $(1, 3, 5, 7, 9)$ and two choices for the distance metric (ℓ_1 and ℓ_2), we would run validation for each of the ten combinations of k and a distance metric, and select the pair that resulted in the smallest validation error.

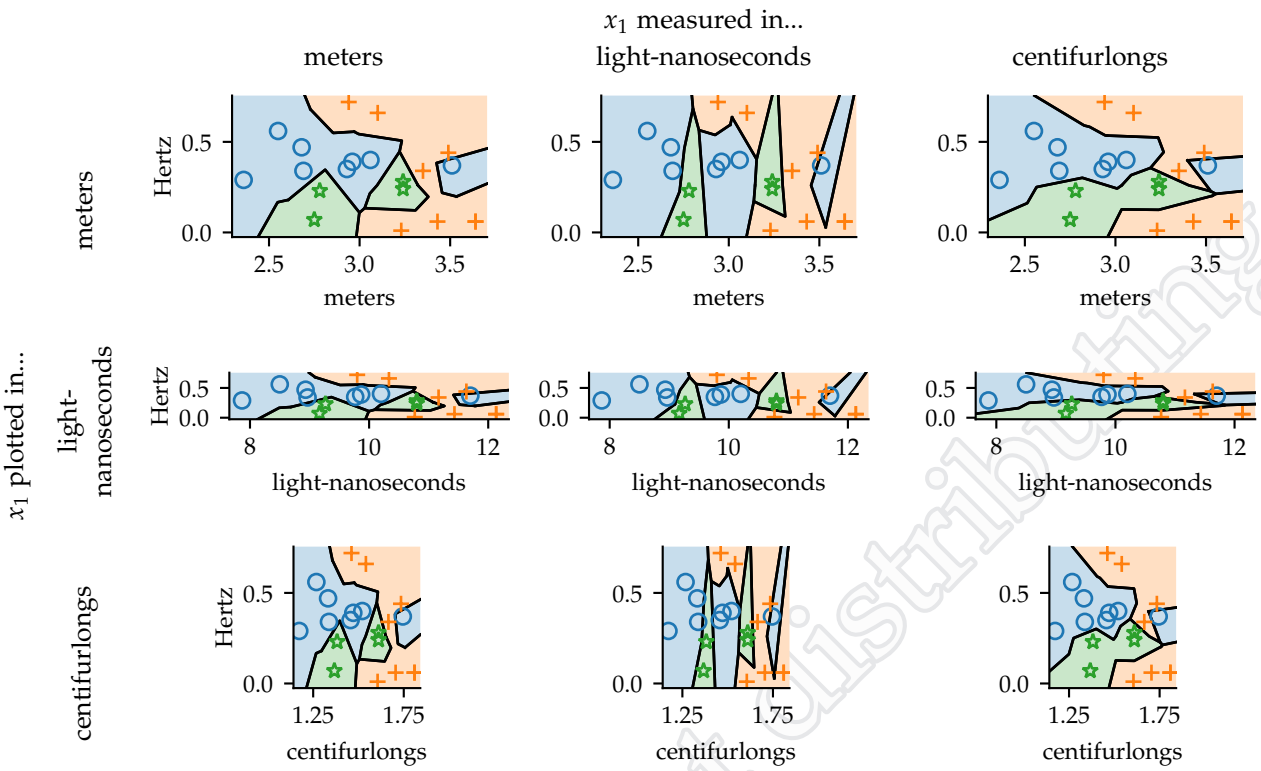

Figure 9.11: Decision boundaries for the same data, but with different axis scalings. See text for explanation and analysis.

Normalization

The choice of the distance metric is not as simple as Euclidean or Manhattan.

Feature Scaling

The data in Figure 9.1 are the girths and ear wiggle rates of 20 hippopotamuses. They are measured in meters and Hertz, respectively. However, they could have been measured in light-nanoseconds[3] and Hertz. Or they could have been measured in centifurlongs[4] and Hertz. We could also change the units of the ear wiggle rate from Hertz to revolutions per minute (for example). Figure 9.11 demonstrates the effects of selecting different units for the girth. The diagonal plots show the nearest-neighbor boundaries for the different unit choices. The off-diagonal plots show these same boundaries rescaled to other units. Therefore, each of the plots in the same column represent the same decision rule. Each of the plots in the same row use the same scaling for the plotting, so that we can compare the differences.

[3] 1 light-nanosecond is the distance light travels in one nanosecond in a vacuum, about 35 barleycorns.

[4] 1 centifurlong is one-hundredth of a furlong, or approximately 238 barleycorns.

So which is the "correct" boundary? There is no definite answer. Changing the units for a feature is just the same as multiplying that feature's values (for all examples in both the training and testing sets) by a scalar value. Any scalar value is possible. The relative scalings of the different features (just two in this case: girth and ear wiggle rate) matter for the final decision surface, but no particular scaling is correct.

If we know a feature to be irrelevant, we should scale it by 0, making this feature the same for all examples and essentially a constant — no decision can be based on this feature. Similarly, if we know a feature to be less important, we should scale it by a small (less than one) value to de-emphasize it. Consider that, in Figure 9.11, when the girth is measured in centifurlongs (which are longer than a meter, and therefore the values for x_1 are now smaller by a factor of 0.497) the decision boundaries depend more on x_2. That is, in the plot, the boundaries tend to run more horizontally, making distinctions based on the vertical axis more. By contrast, when measuring x_1 in light-nanoseconds (multiplying x_1 by 3.33), the boundaries tend to run more vertically because the resulting decision function depends more on x_1 than on x_2.

As discussed previously, if k is chosen properly as a function of the number of data points, m, then k-nearest neighbor will converge to the Bayesian optimal classifier as $m \to \infty$. This is true regardless of the distance metric or scaling (which is just another aspect of the distance metric). However, for finite amounts of data, the effectiveness of k-nearest neighbor can depend greatly on the feature scalings chosen.

Key point
The feature scalings are just another aspects of the distance metric chosen, explicitly or implicitly.

Feature scaling matters for other methods as well. Linear methods, such as linear regression and logistic regression, are invariant to scaling of features. However, once regularization is added, the scaling of the features becomes very important. The performance of non-linear methods (such as neural networks) are usually dependent on the feature scaling.

Scaling Selection

If there is no obviously correct way to scale the features, how can we select a scaling? We could treat the scalings as parameters and seek to tune them to optimize the loss. This approach is known as distance metric learning and can be made even more general to learn other aspects of the distance metric. In the extreme, if provided enough flexibility, learning the best distance metric is equivalent to learning the classifier itself: If the learned distance puts two points of the same class at distance 0 and two points of different classes at any

distance greater than 0, it has solved the learning problem.

We focus here on methods for fixing just the feature scaling and without considering the labels or targets in the training data. We will revisit feature scaling again in Chapter 12.

If you have prior knowledge about the features, that should inform the proper scaling. For example, if a group (or all) of the features measure the same type of information in the same way, it might make sense to use the same scaling for all of these features. This can happen if the features are all the values the pixels in an image, or all ratings of movies on a standard scale, or all durations of time between hippopotamus sightings. If there is no specific reason to prefer one feature to another (they are homogeneous in some fashion), using the same scale for all may make sense.

However, be careful. Just because they are originally in the same units does not mean they should have the same scaling. For example, if classifying images into "portrait" or "non-portrait," the pixels in the middle of the image are probably more important because portraits are of people and they tend to be framed so that the person is in the middle of the image (whereas the pixels around the border of the image are probably much less informative). Similarly, if predicting whether a hippopotamus likes jazz, their rating of certain movies may be more informative than other movies.

Barring such prior knowledge of the relationship of the features to the learning task, we usually resort to trying to make all of the features equally important as inputs and let the core machine learning method sort out which are most related to the targets. The question is, then, "what does 'equally important' mean?" The importance of a feature *as an input* is often taken to be related to its variation. That is, a feature that has little variation (all of the values are approximately the same) *appears* to not be important. In a linear function, the associated weight would need to be much larger for such a feature, compared with a feature with large variation. Because regularization tends to penalize large weights, a feature with small variation is less likely to play a role in the final learned function.

There are two primary methods used in machine learning to scale features to all have the same variation. The first is **range normalization** Each feature is scaled to take on the same range. Here we have

chosen the range -1 to $+1$, although 0 to 1 is also common:

$$\underbrace{a_j}_{\text{minimum value of feature } j} = \min_{i=1}^{m} x_{i,j}$$

$$\underbrace{b_j}_{\text{maximum value of feature } j} = \max_{i=1}^{m} x_{i,j}$$

$$\underbrace{x_{i,j}}_{\text{new value of feature } j \text{ on example } i} \leftarrow 2\frac{\overbrace{x_{i,j}}^{\text{old feature value}} - a_j}{b_j - a_j} - 1 \tag{9.6}$$

Further information
In some contexts, range normalization is just known as min-max normalization or just normalization.

For the jth feature, its value in the ith example is replaced, using this last formula. To calculate this, we first calculate the minimum value, a_j, and maximum value, b_j, that this feature takes on across all examples.

Key point
The feature minimums and maximums from the training set must be remembered for testing.

It is important to store a_j and b_j for all features so that when a testing example, $\vec{x}$, is presented, its features can be transformed in the same fashion: $x_j \leftarrow 2\frac{x_j - a_j}{b_j - a_j} - 1$. The function was learned using features that had been transformed with exactly this scaling. All future uses of the function for testing need to also perform this scaling in the same way.

Range normalization works well for features with well-defined ranges. Ratings of movies on a scale of 0 to 5 and grade level in school are examples. In many of these cases, the minimum and maximum values for the features are known ahead of time. Range normalization does not work well when feature may have outliers. A single very large or very small value will dictate the scaling used even though it may not be indicative of the typical range of the feature.

It is more common, therefore, to use **z-score normalization** where the feature is shifted and scaled so that, in the training data, it has zero mean and unit standard deviation:

Further information
Z-score normalization goes by many names including standardization and z-normalization. The resulting feature is referred to as a z-score.

$$\underbrace{\mu_j}_{\text{mean value of feature } j} = \frac{1}{m}\sum_{i=1}^{m} x_{i,j}$$

$$\underbrace{\sigma_j}_{\text{standard deviation of feature } j} = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(x_{i,j} - \mu_j\right)^2}$$

$$\underbrace{x_{i,j}}_{\text{new value of feature } j \text{ on example } i} \leftarrow \frac{\overbrace{x_{i,j}}^{\text{old feature value}} - \mu_j}{\sigma_j}\,. \tag{9.7}$$

This does *not* transform the features to have a normal distribution. It merely translates and scales them to all have the same mean (0)

and standard deviation (1). Again, the values of μ_j and σ_j from the training set must be retained for all features so that any future testing point can be transformed in the same way.

Because z-score normalization roughly adjusts the features to have the same variation, it also often makes machine learning algorithms more numerically stable and computationally efficient. Floating point arithmetic is more accurate when the values involved are the same orders of magnitude. Further, algorithms like gradient descent are more efficient when the loss function to be minimized has roughly the same curvature in each direction. While it is not obvious (nor guaranteed) that z-score normalization will produce this, analysis beyond the scope of this text can show that it usually does.

Normalization is so common and helpful for practical results that when describing techniques, it is often omitted from the description even though it might be critical for the success. Most common machine learning code libraries have routines to normalize data and many times their implementations of supervised learning methods will include normalization as an implicit initial step. Reading the documentation carefully can usually determine whether you should normalize first or whether the method will do it on your behalf.

Further information
Check any machine learning libraries you use to determine if they implicitly perform normalization.

High-Dimensional Spaces

This text has examples shown in two (or occasionally three) dimensions because that is what our visual systems are used to processing. However, most machine learning problems have more than two or three features, and therefore, the data "live" in a higher dimensional space. Our intuitions from our three-dimensional physical space may lead us astray when reasoning about such high dimensional spaces.

Consider data that has been normalized so that all features are between 0 and 1. For two-dimensional data, all of the data points are inside a square. For three-dimensional data, they are all in a cube. For higher dimensions, they are in a hypercube. If we take a point in the middle (an "average point"), consider its distance to any of the faces of the (hyper)cube and compare that to its distance to any of the corners in the (hyper)cube (Figure 9.12). The first is how far away another point can be by varying only one feature. The second is how far away another point can be by varying all of the features. The distance to the face is $\frac{1}{2}$. The distance to the corner is $\frac{1}{2}\sqrt{n}$. Thus, the distance to the corner is $\sqrt{n}$ times larger than the distance to the face. As the dimension of the data, n, grows the difference grows. The corners get farther away. In high dimensions, most of the volume of a hypercube is in the corners and all of the corners are far away from each other.

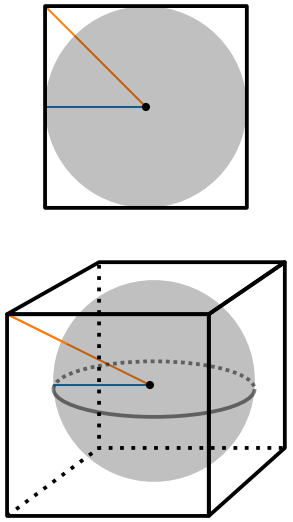

Figure 9.12: Square (top) and Cube (bottom). Diagonal, orange line is the distance to a corner. Horizontal, blue line is the distance to the closest boundary point. As the dimension increases, these distances differ more. Gray represents all of the points within the blue distance. As the dimension increases, this is a decreasing fraction of all of the volume.

Additionally, as n grows, the number of corners grows. There are 2^n corners of an n-dimensional hypercube. More and more of the volume of the hypercube is near the corners. The geometry of a hypercube is more like that of a spiky ball than of a cube: many corners, each far apart from each other.

For algorithms like nearest neighbor that rely on locality for their estimation, this can be a problem. We assume that some points will be near the test point and therefore more relevant than other points that are farther away. However, if the training data are uniformly distributed throughout the space, as the dimension increases, the distance to the closest training point increases, and (more importantly) all of the training points are approximately the same distance away. This means that there is not a clear notion of a "nearest neighbor."

Put differently, this means that as the dimensionality of the data (number of features) increases, we need more and more data to "fill the space," particularly for an algorithm like nearest neighbor. In general, the data requirements grow exponentially with the dimensionality. This is a manifestation of a general problem known as the **curse of dimensionality**. As we increase the number of features, the number of ways in which those features might interact or combine to affect the prediction target grows exponentially. For a classifier as flexible as nearest neighbor, this poses a problem because it needs exponentially more data to determine which of these possible interactions explain the data.

Let ϵ represent the distance at which we would consider points to be similar. We would like testing points to be at least this close to the nearest training point for us to trust nearest neighbor. Figure 9.13 shows how many training points we would need for this to be true on average (the number to force this to be true with high probability is much larger). Even for only a few features, massive amounts of training data are necessary. With only ten features, even if we are willing to accept $\epsilon = 0.5$ (half the variation in any single feature!) as close, we need a thousand training data points. If we pick $\epsilon = 0.1$, that number climbs to two billion data points. As the number of features increases, the number of needed data points grows geometrically.

So what to do? The most obvious choice is to try to keep the number of features small. But, if this is not possible, we can select a method that does not try to model all possible combinations of feature interactions. Consider a linear classifier: It has only n parameters. It assumes that each feature has its own effect on the prediction, but that we do not need to consider pairs, triples, or other combinatorial effects. A linear classifier needs only a number of data points that scales linearly with the number of features. While we might miss

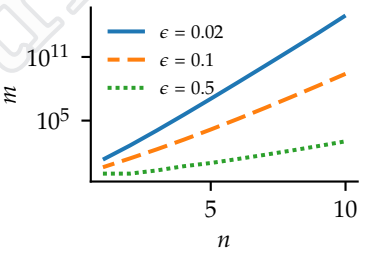

Figure 9.13: Minimum number of training data points (m) needed to ensure a testing point (on average) will be within ϵ of the nearest training point, as a function of the number of features (n), assuming features are uniformly distributed between 0 and 1). Note the vertical axis is on a logarithmic scale.

out on pairwise feature-feature interactions, we gain in our ability to reliably estimate the parameters (weights) of our function.

One other aspect of most data might save us: Typically the feature values are not independent. The analysis and plot above assumes that the data are distributed evenly over the hypercube. However, in most problems the features are related to each other. If the features ($\vec{x}$) are related to the target (y), it would make sense they are also related to each other. For instance, we expect the hippopotamus's length and weight to be roughly correlated. What matters to an algorithm like nearest neighbor is the **effective dimensionality** or **intrinstic dimensionality**[5] of the data: the number of ways in which the data might vary. Figure 9.14 shows a dataset with three features, but it has an effective or intrinstic dimensionality of two. There is a lower-dimensional manifold which contains the data (or which the data are approximately near). If this lower-dimensional manifold is linear, we can often efficiently find it and project the data to this subspace (see Chapter 12). One advantage of nearest neighbor is that it can exploit this lower effective dimensionality without explicitly finding the manifold, if such a manifold exists.

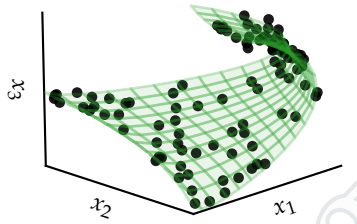

Figure 9.14: Data that resides on a lower dimensional manifold of the full feature space. The green manifold is not known, but the black data points lie approximately on this manifold.

[5] Effective and intrinstic dimensionalities are different, but similar enough for the purposes of this discussion.

Computational Considerations

While the data necessary to fuel nearest neighbor classification may be difficult to obtain for high-dimensional problems, the computational power necessary can be equally problematic. Nearest neighbor is a lazy method and so all of the computation comes at testing (or querying) time. This is the reverse of what we typically want: Usually, we have time upfront (during training) to ponder the data and determine the rule. When faced with a new hippopotamus, we would like to be able to quickly predict whether it is aggressive.

If we make nearest neighbor less lazy, we might be able to achieve this. For example, if the problem is one-dimensional (only a single feature), we can build a binary search tree of the training data's x values at training time and use this at testing. At testing or querying time, if we follow the path to insert the query point into the search tree (but do not actually insert it!), the path will contain the nearest neighbor in the training set. Figure 9.15 demonstrates this for a simple example. While scanning the original training data would take $O(m)$ time, scanning a balanced binary search tree would take only $O(\ln m)$ time. This would greatly reducing the testing time for nearest neighbor.

This one dimensional search tree divides points by whether they are "less than" or "greater than" particular points chosen to be the internal nodes of the tree. There are a couple of different ways to

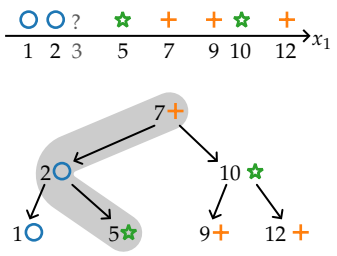

Figure 9.15: An example of a binary search tree for data with only a single feature. The path taken when searching for where to insert a new point (without actually inserting it) will pass through a node that contains the nearest neighbor. This is shown for the example query point $x_1 = 3$, where the nearest neighbor ($x_1 = 2$) is along the path.

extend this idea to more dimensions. A **quadtree** and its generalizations also place the training points into a tree. However, because there are multiple features, there are multiple dimensions along which a point might be "less than" or "greater than" another point. Thus, there are more children for an internal node in the tree, corresponding to the different comparisons. In two dimensions (a quadtree), each point in the tree has four children for the four different possibilities for two binary comparisons. In three dimensions (an **octree**), each node in the tree has eight children. Beyond three dimensions, this type of data structure does not work well in machine learning because of the large number of children for each internal node.

Instead of making a comparison in every feature (or dimension) at internal nodes, each internal node can compare just based on one feature. Typically, the feature on which to compare is cycled through the possible features based on the depth node in the tree. For example, if there are three features, the root node would split based on x_1, all of its children would split based on x_2, all of their children would split based on x_3, and then all of *their* children would split based on x_1 and the cycle repeats. This is called a k-dimensional tree, or just ***k*-d tree**.[6]

An example of a k-d tree is shown in Figure 9.16. For this tree, the top level (node A) splits on x_1, the next level (nodes B and C) splits on x_2, and the last level (nodes D, E, F, and G) splits on x_1. Critically, note that the search path for a query point (shown with a "?") does *not* pass through the closest point. This is, unfortunately, unavoidable in higher dimensions, even with other types of search trees. ***k*-d trees** can be used to speed up nearest neighbor queries. However, more of the tree must be searched. The resulting algorithm is a branch-and-bound style search where the entire tree must be searched, but some subtrees may be pruned along the way based on the closest point found so far. This method can be modified to return the set of $k \geq 2$ nearest neighbors.

In the worst case, no branches of the k-d tree can be pruned and nearest neighbor search takes $O(m)$ time, the same as just scanning the original training data. In practice, for smaller dimensional problems ($n < 10$), k-d trees can often work well. With more features, a linear scan of the training data is usually faster because of its simplicity. Often an approximate nearest neighbor is deemed "good enough." There are a range of approximate nearest neighbor algorithms, but that is beyond this chapter's scope.

Further information
Quadtrees and octrees are more commonly used in computer graphics where the dimensionality is typically two or three.

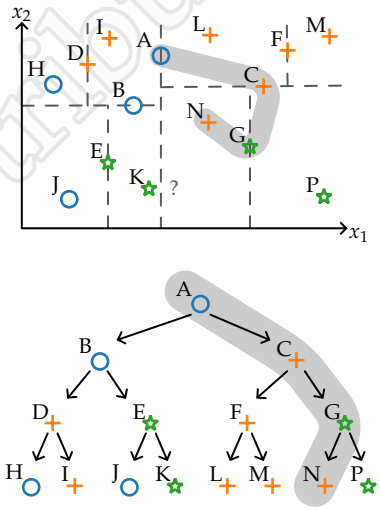

Figure 9.16: An example of a k-d tree for data with a two features. The points are labeled with letters for correspondence between the data (top) and the tree (bottom). Unlike the one-dimensional case, the path taken when searching for where to insert a new point (without actually inserting it) will *not* necessarily pass through a node that contains the nearest neighbor! This is shown for the example query point (?), where the nearest neighbor is point K, not on the path.

[6] Yes, once again the variable k is being used for something different. This k is not the same as the number of nearest neighbors or the number of folds of cross-validation.

Exercises

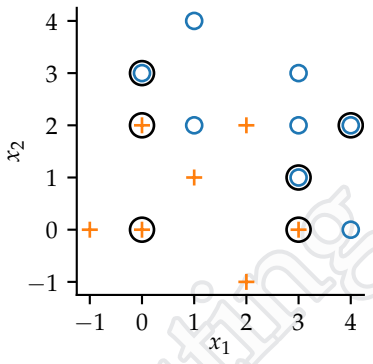

Figure 9.17: Toy nearest-neighbor dataset. Circled points are part of the validation set.

Exercise 9.1

Consider the binary classification dataset in Figure 9.17. The data have been split into a training set (9 points) and a validation set (6 points). The validation points are circled. The non-circled points are the training set.

Choose between 1-nearest neighbor and 3-nearest neighbor (both with the Euclidean distance metric) using holdout validation. Which would be selected?

Exercise 9.2

Using the binary classification dataset in Figure 9.17 and treating all points as part of the training set (ignore the black circles around some points), choose between 1-nearest neighbor and 3-nearest neighbor (both with the Euclidean distance metric) using *leave-one-out cross-validation*. Which would be selected? Break ties in any way you see fit.

Exercise 9.3

Using the binary classification dataset in Figure 9.17 and treating all points as part of the training set (ignore the black circles around some points), choose between the two scalings below (with the Euclidean distance metric) for nearest neighbor ($k = 1$) using *leave-one-out cross-validation*. Which would be selected? Break ties in any way you see fit.

Scaling I: multiply x_1 by 10, multiply x_2 by 1.

Scaling II: multiply x_1 by 1, multiply x_2 by 10.

Exercise 9.4

Assume the finding a nearest neighbor is done with a linear scan (that is, there is no use of a kd-tree or similar data structure). Further assume that the calculation of a distance between two points dominates the computational time necessary. Let there be m data points in the full training set.

a. How many comparisons are necessary to evaluate one hyperparameter setting using leave-one-out cross-validation?
b. How many comparisons are necessary to evaluate one hyperparameter setting using 10-fold cross-validation?
c. When is leave-one-out cross-validation more computationally expensive than 10-fold cross-validation?

Exercise 9.5

a. Write code that takes as input a set of points, $\boldsymbol{X}$, and calcu-

lates the average distance for each point to its nearest neighbor in the same set.

b. Write code that takes as input a set of points, $\boldsymbol{X}$, and calculates the average distance between any pair of points.
c. Write code that calculates the ratio of these two quantities (the average distance to the closest point divided by the average distance to an average point), averaged across 5 random draws of m points with n features (dimensions) where each feature is chosen at random from a uniform distribution.
d. Plot the result as a function of n for $n \in \{2, 4, 8, 16, 32, 64\}$ for different values of m, $m \in \{10, 100, 1000, 10000\}$.
e. What are the implications for nearest neighbor?

Further information
You can try other distributions for the features. You will get similar results for almost any other distribution if each feature is chosen independently.

Exercise 9.6

The goal of this exercise is to derive the formula for the worse-case asymptotic error for ($k = 1$) nearest-neighbor as a function of the Bayes-optimal error rate. The worst case occurs when the Bayes-optimal error rate is the same for any $\vec{x}$. That is, no matter what $\vec{x}$ is, the best possible $f(\vec{x})$ will classify it incorrectly that same fraction of the time. You do not need to prove this.

a. Consider binary classification $C = 2$. If the Bayes-optimal error rate is b, prove that the asymptotic error rate of nearest neighbor is $2(b - b^2)$.
b. What is the worst-case asymptotic error rate as a function of the Bayes-optimal error rate, b, when $C > 2$?
c. Show that your answer to part b is no greater than twice b.
d. Show that your answer to part b is no greater than 0.25 plus b.